Monitoring ML and AI Model Inference and Performance

Slide 01: Title

Welcome back to the theory-only learning series for the AWS Certified Machine Learning Engineer Associate exam. In the last three videos we finished Domain 3, where we deployed models and automated their lifecycle. Now we start Domain 4: Operating, Monitoring, and Securing ML and AI Solutions. This is Domain 4, Part 1, and the focus is monitoring. A model that worked well on launch day can quietly get worse as the world changes. So in this video we look at what goes wrong in production, how SageMaker Model Monitor detects drift, how to test live models safely, how to catch inference errors, and how to monitor agents and foundation models. Let's get started.

Slide 02: Production models degrade over time

Once a model is in production, three kinds of problems show up. The first is data drift. The inputs the model receives start to look different from the training data. Maybe the business entered a new market, or customer behavior changes with the seasons, or an upstream team changed how a field is calculated. The second is concept drift. Here the inputs might look the same, but the right answer has changed. Fraud is the classic example: fraudsters change tactics, so patterns that used to be safe become risky. The third is pipeline errors. The model itself may be fine, but missing features, a schema change, or a timeout breaks the predictions. On the exam, if accuracy drops slowly over time, think drift. If predictions fail suddenly, think pipeline errors.

Slide 03: SageMaker Model Monitor detects drift

Amazon SageMaker Model Monitor follows a simple loop. First, capture. You turn on data capture for the endpoint, and SageMaker AI saves a sample of requests and responses to Amazon S3. Second, baseline. A baselining job looks at your training data and creates statistics and constraints, such as expected data types, value ranges, and how often values are missing. Third, schedule. You create a monitoring schedule, often hourly or daily, that analyzes the captured data. Fourth, compare. Each run checks the live data against the baseline and writes a violations report when something breaks a constraint. Fifth, alert. Model Monitor publishes metrics to Amazon CloudWatch, so you can set alarms. And as we saw in Domain 3, that alarm can trigger an automated retraining pipeline. This loop is a favorite exam topic, so remember the order: capture, baseline, schedule, compare, alert.

Slide 04: Model Monitor has four monitor types

Model Monitor offers four types of monitors. Data quality monitoring checks the input features against the baseline. It catches missing values, wrong data types, and shifts in distribution. Model quality monitoring compares the model's predictions against ground truth labels, using metrics like accuracy, RMSE, or F1 score. The catch is that you need those labels, which often arrive later, for example when you finally learn whether a loan was repaid. This is the monitor that reveals concept drift. Bias drift monitoring uses SageMaker Clarify to track fairness metrics for different groups over time. And feature attribution drift uses Clarify SHAP values to check whether the features driving predictions have changed. If a feature that used to be unimportant suddenly dominates, something has changed. Exam tip: if the question mentions ground truth, the answer is model quality monitoring.

Slide 05: Test new models on live traffic safely

Offline evaluation is not enough. Real traffic is the true test, so SageMaker AI gives us two ways to test models on live traffic. On the left is A/B testing with production variants. One endpoint can host several variants, and variant weights decide how much traffic each one receives. For example, ninety percent to the current model and ten percent to the new one. You can also call a specific variant with the TargetVariant parameter. CloudWatch reports metrics per variant, so you can compare them side by side and gradually shift the weights. On the right is shadow testing. Here the new model receives a copy of production requests, but its responses are never returned to users. You compare latency, errors, and outputs with zero risk to customers, then promote the shadow model when it performs well. Exam tip: if users must not be affected at all, choose shadow testing.

Slide 06: Catch errors in inference pipelines

When predictions fail, we need to find out fast. SageMaker AI endpoints publish metrics to Amazon CloudWatch. Invocation four-X-X errors are client-side problems, such as a badly formatted request. Invocation five-X-X errors are server-side problems inside the model or container. Model latency is the time the model itself takes, while overhead latency is the time SageMaker AI adds around it. Comparing the two tells you whether the model or the infrastructure is slow. Metrics tell you something is wrong, and logs tell you why. Each container writes to CloudWatch Logs, where you will find stack traces, input parsing failures, and out-of-memory errors. Finally, CloudWatch alarms watch the metrics, and Amazon EventBridge rules plus Amazon SNS notifications route problems to the right people or to automated fixes. Exam tip: four-X-X means check the request, five-X-X means check the model and container.

Slide 07: Agents need step-by-step observability

Agents are harder to monitor because a single request can involve many steps. Amazon Bedrock Agents can return a trace that shows each step: how the agent reasoned, which knowledge base it searched, and which action group it called. Amazon Bedrock AgentCore

Observability goes further. It is built on OpenTelemetry and shows sessions, traces, and spans in Amazon CloudWatch, including the latency of each step and the tokens used per session. Then we watch for failure signals. Tool call errors happen when an API behind an action fails or receives bad parameters. Truncated streams happen when a response is cut off, often because it hit the maximum token limit. And loops or timeouts happen when an agent keeps retrying without making progress. When an agent gives a wrong answer, the trace is the first place to look, because it shows exactly which step went wrong.

Slide 08: Monitor foundation models in Amazon Bedrock

For foundation models in Amazon Bedrock, there are three monitoring tools. First, model invocation logging. When you turn it on, Bedrock records the full request and response for each call to CloudWatch Logs or Amazon S3. It is off by default, so remember to enable it if you need auditing or debugging. Second, runtime metrics. Bedrock publishes metrics to CloudWatch, including invocation latency, input and output token counts, and invocation throttles. Throttles tell you that you are hitting your quota and may need more capacity, such as Provisioned Throughput. Third, Amazon Bedrock evaluations. As we saw in Domain 2, evaluation jobs can score responses with automatic metrics, with a model acting as a judge, or with human reviewers. They also support evaluating RAG systems. Running evaluations regularly on sampled production prompts helps you spot quality drops before your users do.

Slide 09: A monitoring checklist

Let's wrap up Part 1 with a checklist. One: capture endpoint data and create a baseline from your training data. Two: schedule data quality and model quality monitors, and add bias and feature attribution monitors when fairness matters. Three: test new models on live traffic with production variants or shadow testing. Four: set alarms on four-X-X errors, five-X-X errors, and latency. Five: trace agent steps and watch for tool call errors, truncated streams, and loops. Six: turn on Bedrock invocation logging and evaluate quality regularly. The big idea is that monitoring is not one tool. You watch the data, the model, and the systems around it. In Part 2, we will look at optimizing cost and performance. Thanks for watching, and see you in the next video.